# Can an LLM replace the IDS analyst?



Automated generation of intrusion detection rules using generative AI in a rule-based IDS deployment

Group 13 · Bigyan Siwakoti · Binay Siwakoti · Irab Baral · Shashwat Acharya

Supervisor: Prof. Bharanidharan Shanmugam · Charles Darwin University · October 2026

# Detection is only as good as its rules

Signature-based IDS: deterministic and inspectable, but blind to anything without a rule

- No rule, no detection — however capable the engine
- Turning an observed attack into a tested rule is manual work: investigate, identify invariants, write, validate, deploy
- Scarfone & Mell (NIST SP 800-94) describe this cycle as taking hours to days of scarce expertise




Today a human analyst performs every step, for every new attack

**52,647** community rules in ET Open, and still whole attack categories went undetected

# Research question

*Can a large language model take over the analyst's role in the alert-handling loop of a rule-based IDS — and what context does it need to do that job well?*



## Build

An agentic triage engine that investigates Suricata alerts, writes detection rules and validates them by replay



## Measure

Every AI-authored rule against ground-truth labels, on two datasets — one the system was never tuned for



## Test context

Experiment 1: triage with alert events only versus full sensor context

Constraints: open-source only · no machine-learning detection · the model authors rules, it never classifies traffic

# Keep the engine, automate the analyst

Three stages — detection stays deterministic, the model works around it

## 1 Detection

- Suricata 8.0.6 replays pcap files
- ET Open ruleset, 52,647 rules
- Benign Monday baseline: 80 documented suppressions → 0 alerts

## 2 Triage

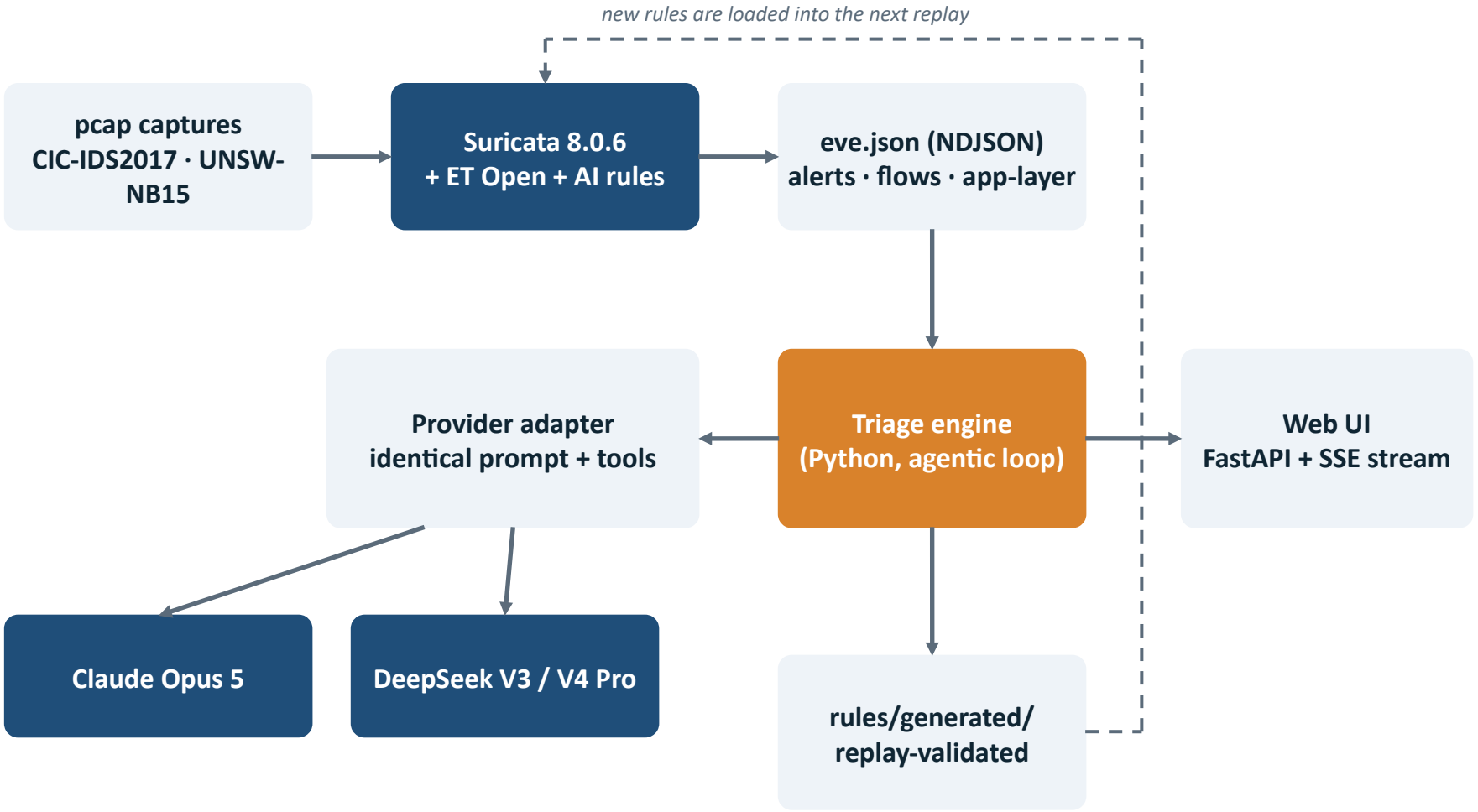
- LLM agent investigates eve.json
- Six-step procedure: big picture → categorise → investigate → deep-dive → act → report
- 8 tools, no free shell access

## 3 Output

- Threat → write rule → validate by replay → auto-deploy
- Benign noise → propose suppression → human approval

Design decisions: alert-only rules (a Suricata 'pass' rule would stop inspecting the whole flow); offline pcap replay; all tuning in threshold.config

# Tech stack and architecture

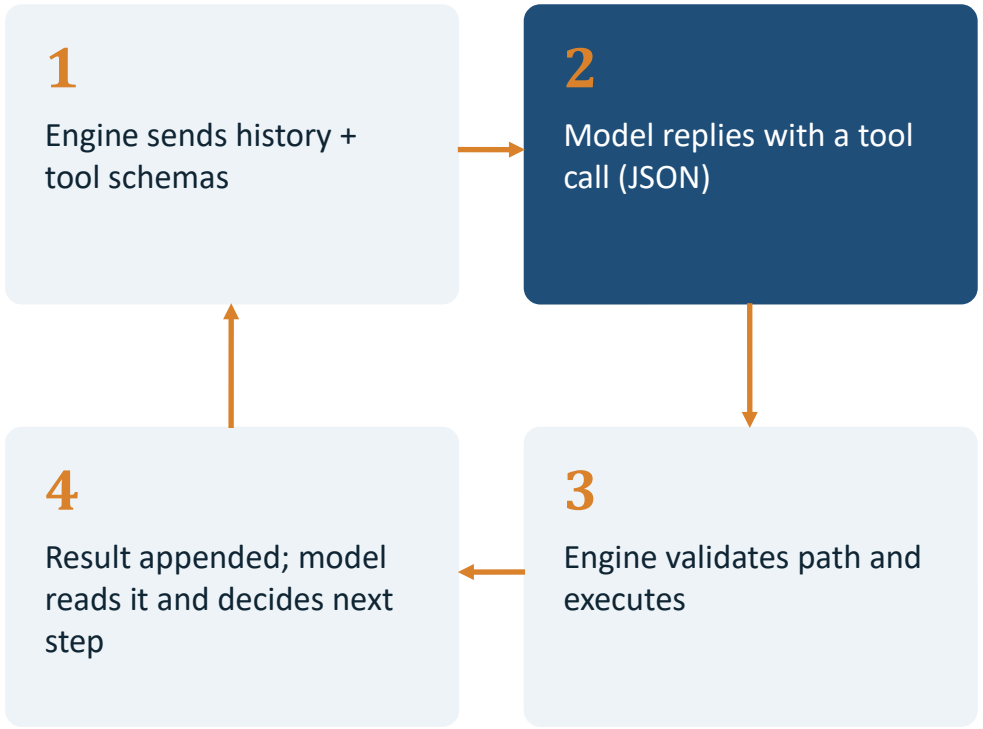



## Stack

- Suricata 8.0.6 (IDS engine)
- ET Open ruleset
- jq over NDJSON
- Python 3 triage engine
- Anthropic + DeepSeek APIs
- FastAPI + Server-Sent Events
- Ubuntu on WSL

# Giving the model agentic capability

A tool-calling loop: the model chooses actions, the engine executes them through validated tools




*repeats until the model answers without a tool call (max 100 turns)*

**`count_alerts`**
alerts per signature

**`count_by`**
aggregate any field

**`query_eve_json`**
jq query, capped output

**`read_file`**
configs, rules, logs

**`write_file`**
rules/generated only

**`append_to_file`**
rules/generated only

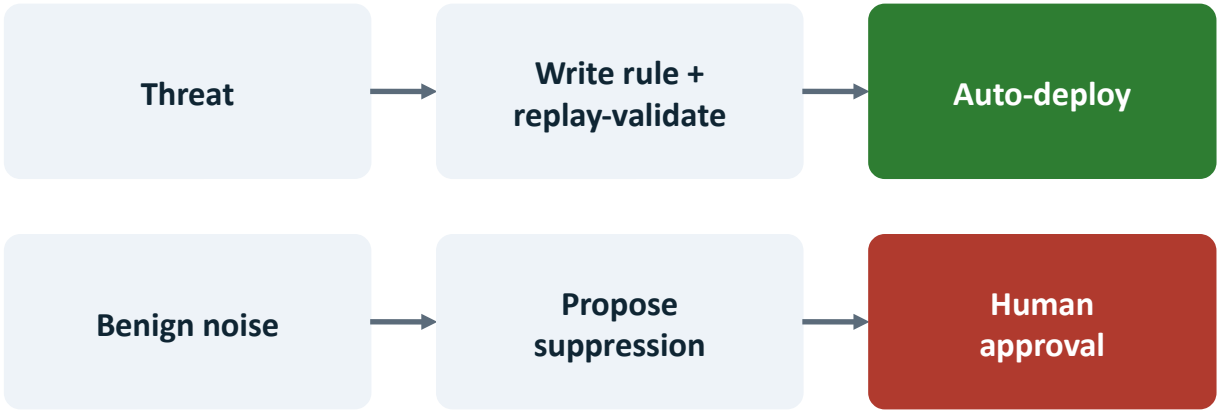
**`run_suricata`**
replay pcap to validate

**`list_directory`**
allowlisted folders

Tool design mattered as much as the model: DeepSeek V3 failed every run until we added count_by

# Safety and evaluation integrity

## Safety asymmetry




A bad rule over-alerts (wrong-way safe). A bad suppression blinds the sensor (wrong-way unsafe) — so a human decides.

It mattered: one UNSW run proposed suppressing rules that were catching a real Cisco heap-overflow exploit. The proposal needed approval, so the sensor stayed sighted.

## Evaluation integrity

**Our transcript audit found label leakage**

- Early runs listed the ground-truth folder — file names like '…PortScan' name the attack
- One Tuesday run read the answer from our own project notes
- Fix: tools can open only logs/, config/, rules/, data/pcaps/
- UNSW runs: zero attempts to read labels

# Two datasets, scored against ground truth

## CIC-IDS2017

Canadian Institute for Cybersecurity

**14**
attack categories

**4**
attack days, tuned + raw passes

**per-flow**
labels (CICFlowMeter)

Brute force, DoS (Hulk, GoldenEye, slowloris, Slowhttptest), Heartbleed, web attacks, infiltration, DDoS, port scan, ARES botnet

## UNSW-NB15

UNSW Canberra Cyber Range Lab

**9**
attack categories

**6**
captures (12 GB), raw only

**31,432**
labelled attack events

Never used during development — tests whether the approach transfers. No benign-only day, so no baseline tuning is possible

# CIC-IDS2017: broad coverage, precise rules

**13 / 14**

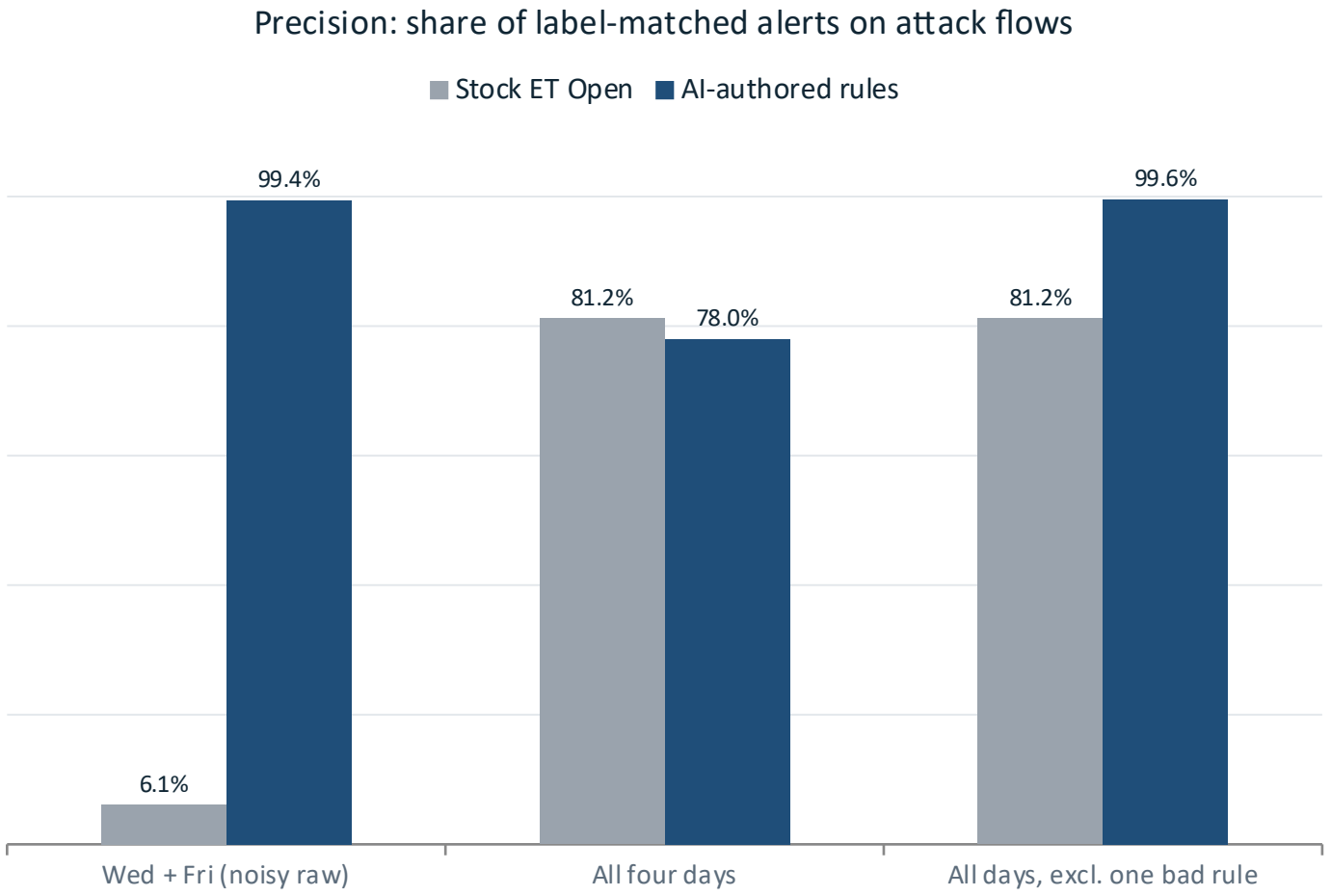
attack categories identified by the model (only DoS GoldenEye missed)

**8 → 12**

of 14 categories caught by rules: stock ET Open → stock + AI-authored rules

**99.4%**

of AI-rule alerts on the noisy days fell on real attack flows (stock rules: 6.1%)




Flow-level matching on 5-tuple and time; Suricata engine events excluded. The one bad rule (FTP 'data staging') fires on routine uploads.

# What the stock rules missed

## FAKEVERB DoS

Diagnosed that Suricata's HTTP parser hides non-standard methods, rewrote the rule on raw TCP payload

**463 hits (stock: none)**

## Hulk GET flood

149,703-request cache-busting flood with no dedicated stock rule; wrote and validated a rate rule

**new detection**

## ARES botnet

Five infected hosts beaconing to a C2 server; no stock or custom alert had fired

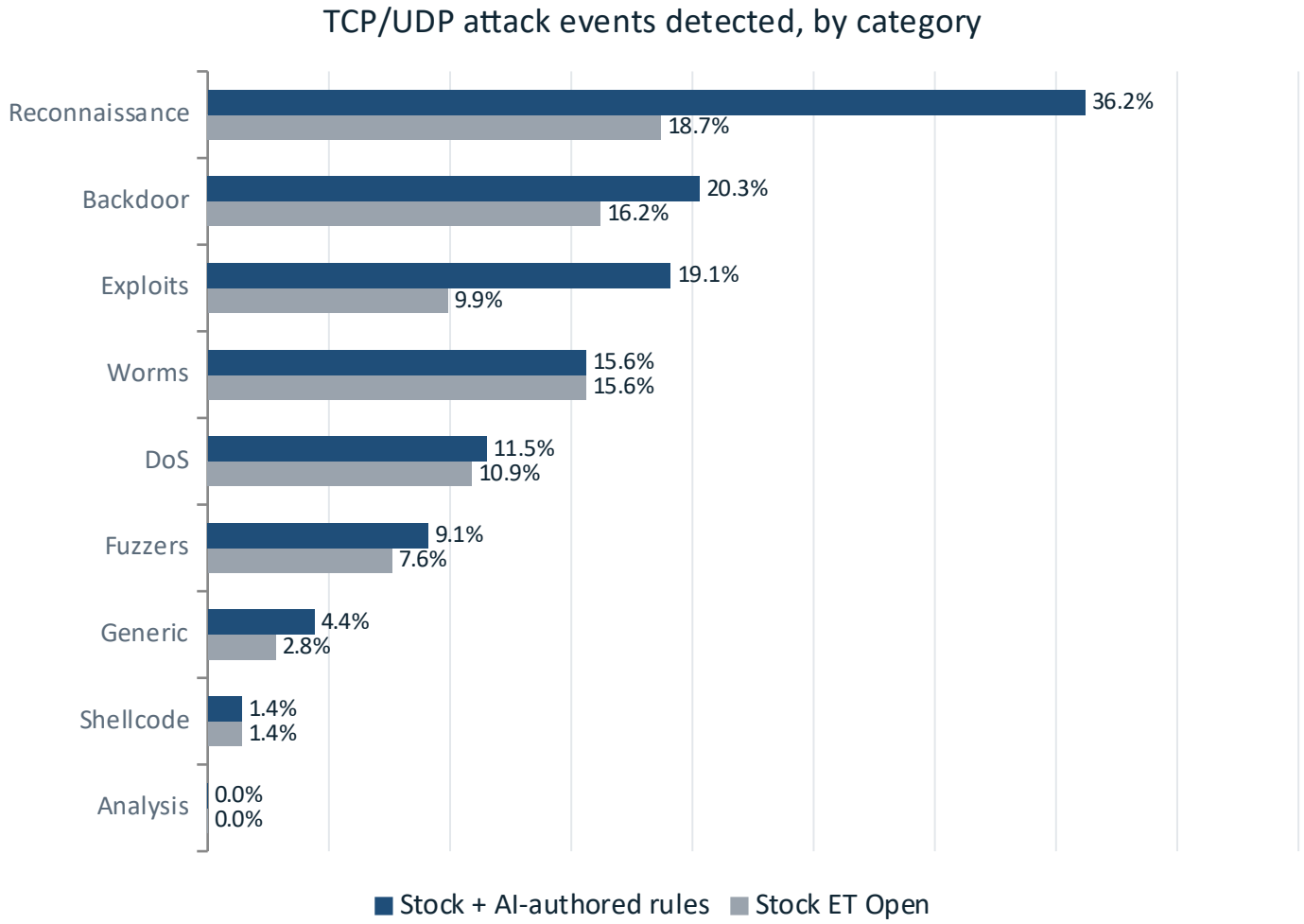
**15 of 15 alerts on Bot flows**

## SSH brute force

Paramiko client fingerprint rule for SSH-Patator

**detection 1.0% → 28.4%, zero false positives**

# UNSW-NB15 v1: the approach works on a second dataset




**9.2% → 15.5%**

event-level detection, stock → stock + AI rules (+68% relative)

**8 / 9**

categories identified across six runs (Analysis missed)

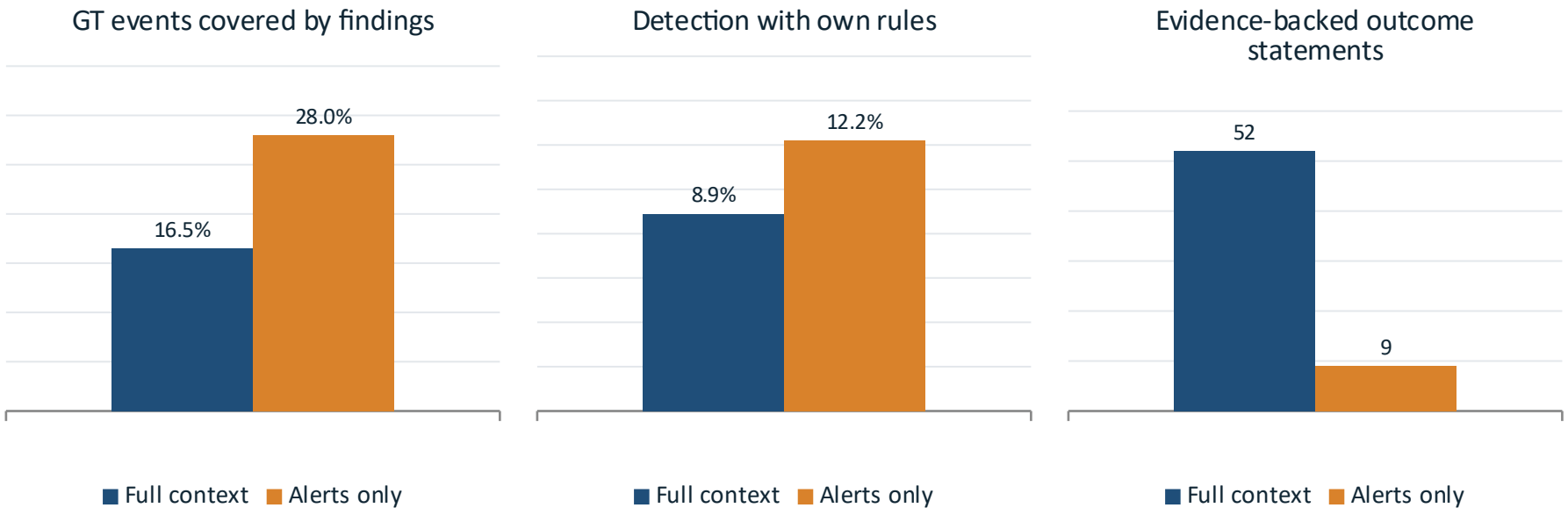
**0**

AI-rule alerts on hosts with no attacker involved

**Within one replay: +5.9 pts genuine gain over stock ET Open. 13 of 15 rules written on other captures fired on capture 14.**

# Experiment 1: does context matter?

Same model and captures (UNSW 3, 1, 10) — full sensor context versus alert events only



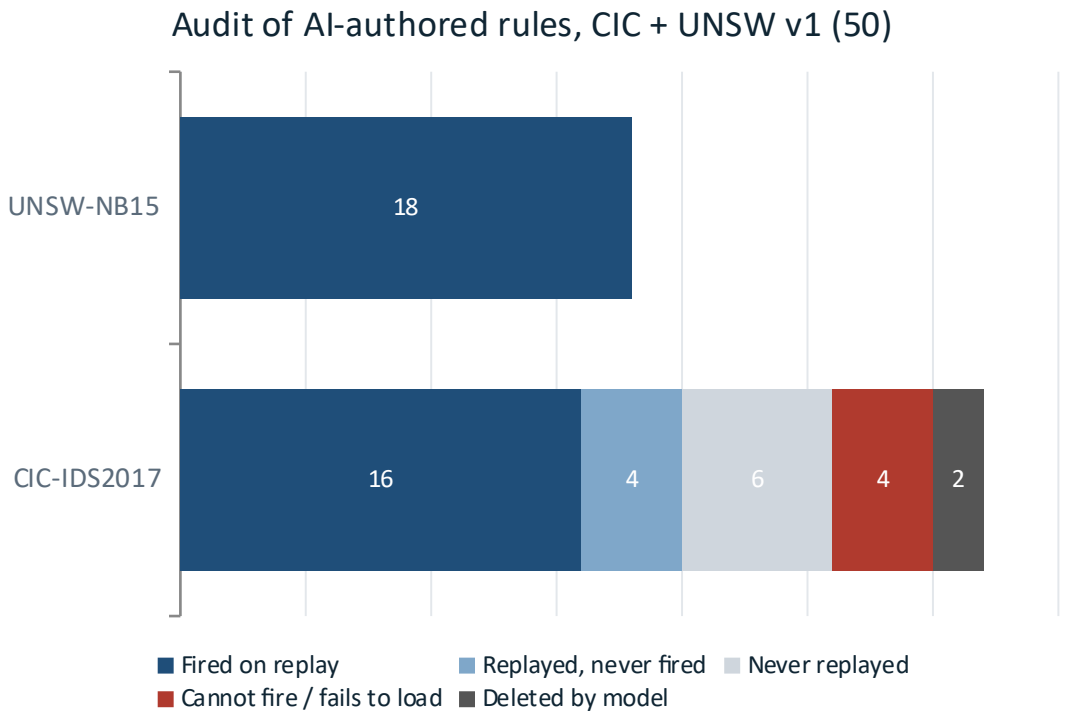

**Breadth without context.**
**Depth with it.**

Only full context let the model answer: did the attack succeed?

- Alerts only covered more categories (7 vs 6) and wrote more productive rules — broad, signature-organised reports
- Full context established outcomes: '59 × PASS → 230 Login successful', '2.25 MB leaked by Heartbleed' — alerts-only reports said success 'cannot be confirmed'

# What went wrong: confident errors

- Deleted two of its best rules, claiming they were stock ET rules — they were not
- Wrote three rules Suricata rejects and one that can never fire
- Read routine FTP uploads from every workstation as data exfiltration
- 'Validated' Thursday rules against a small test file, not the Thursday capture
- Same evidence, opposite verdicts across runs (5 : 1 on the Heartbleed rules)



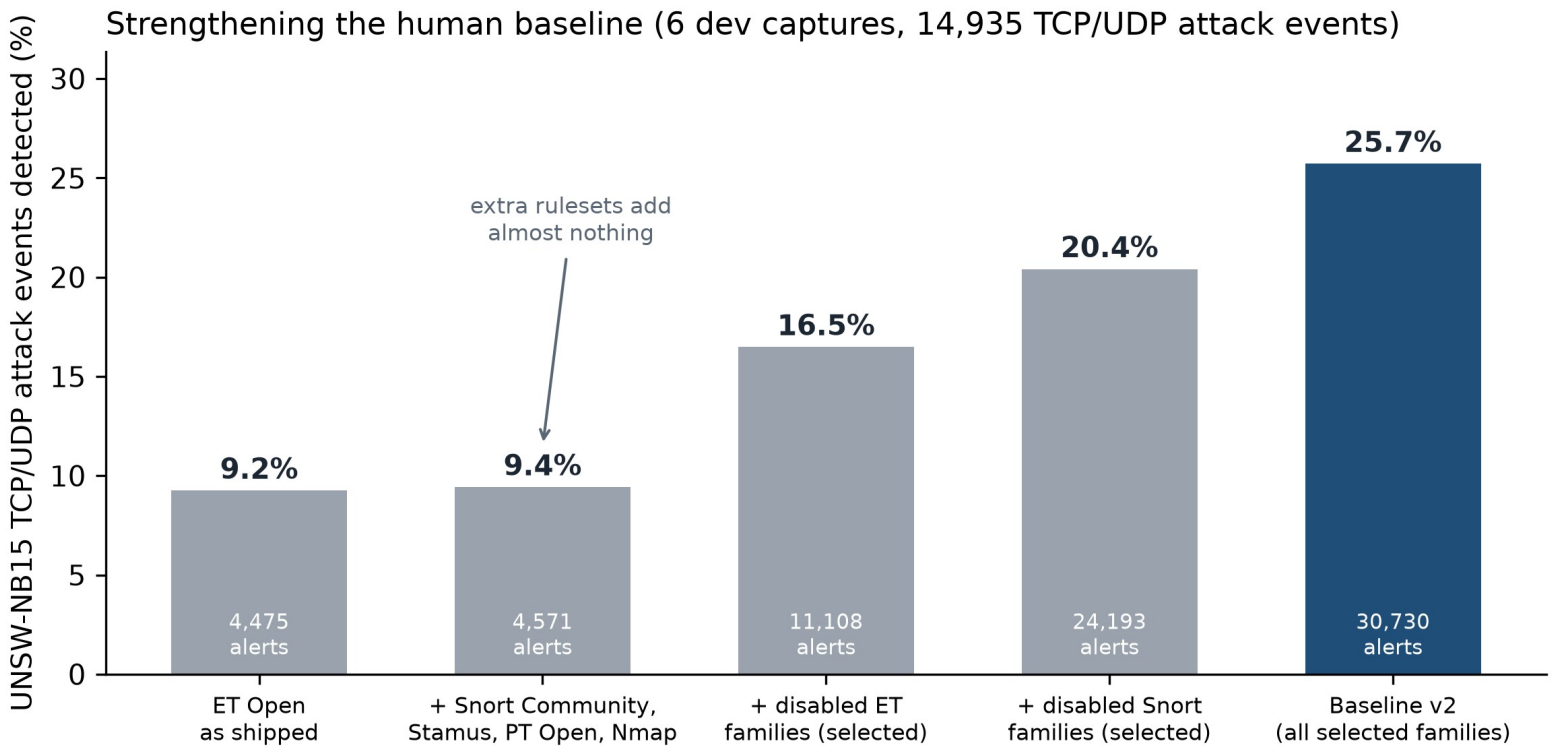

Self-validation is only as good as the environment: CIC's config loaded 3 of 10 rule files; UNSW's loaded the model's file — and all 18 rules work.

# Part 2 — Can the agent beat a strong human baseline?

The supervisor's challenge: look at what real detection engineers do, and replicate it with the agent

# First, make the human baseline as strong as possible

Re-enabling disabled ET and Snort rule families, chosen from the services seen in the traffic — never from labels




**9.2% → 25.7%**

UNSW TCP/UDP attack events detected: ET Open as shipped → baseline v2

**46×**

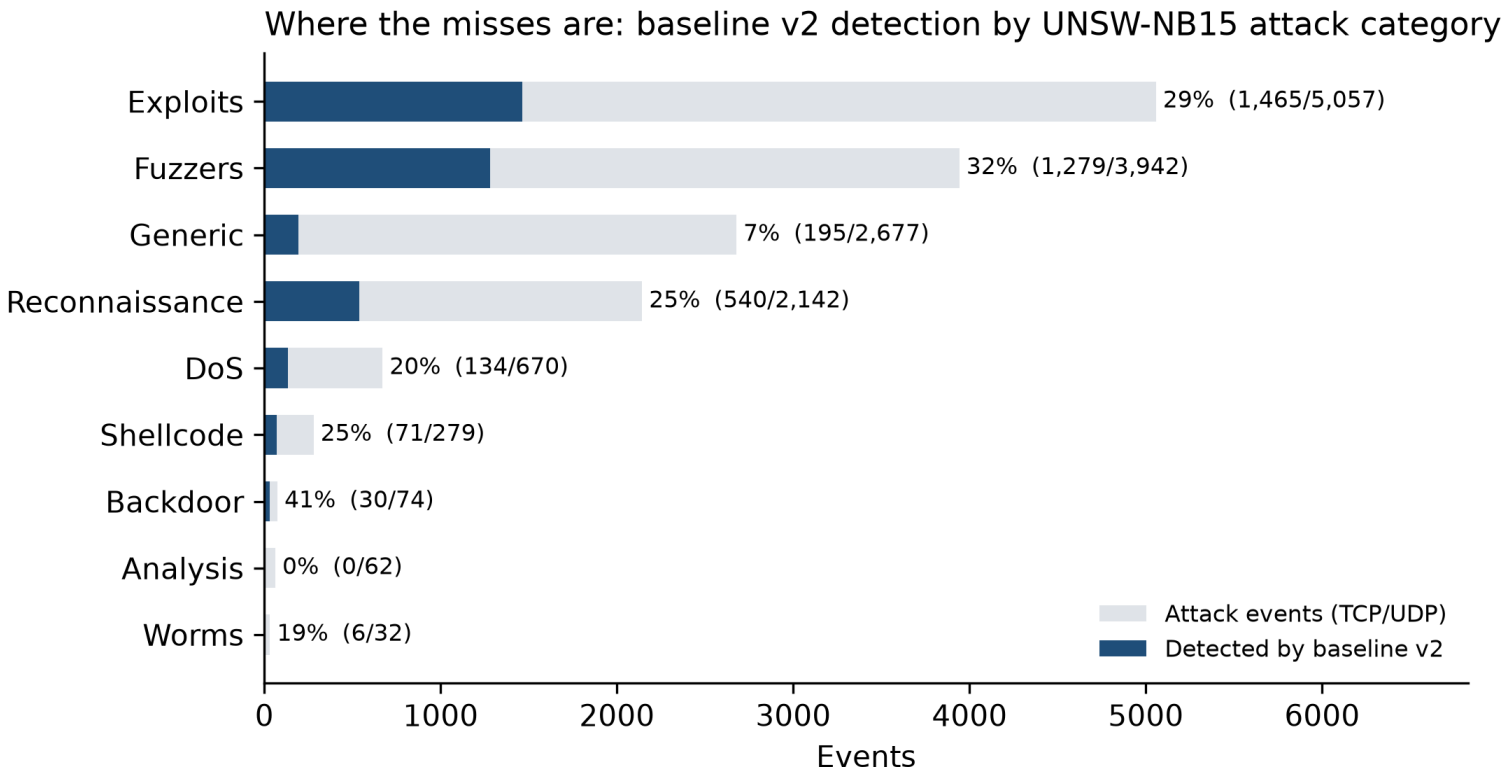
more alerts on non-attacker hosts if every disabled rule is enabled (31.2% vs 24.9% detection)

This is the bar the AI now has to clear — no longer stock rules.

Six development captures (17 Feb 2015), 14,935 TCP/UDP attack events. Extra rulesets: Snort Community, Stamus lateral, PT Open, Nmap (Nmap later dropped: it wrecked CIC precision).

# Where a signature IDS still misses

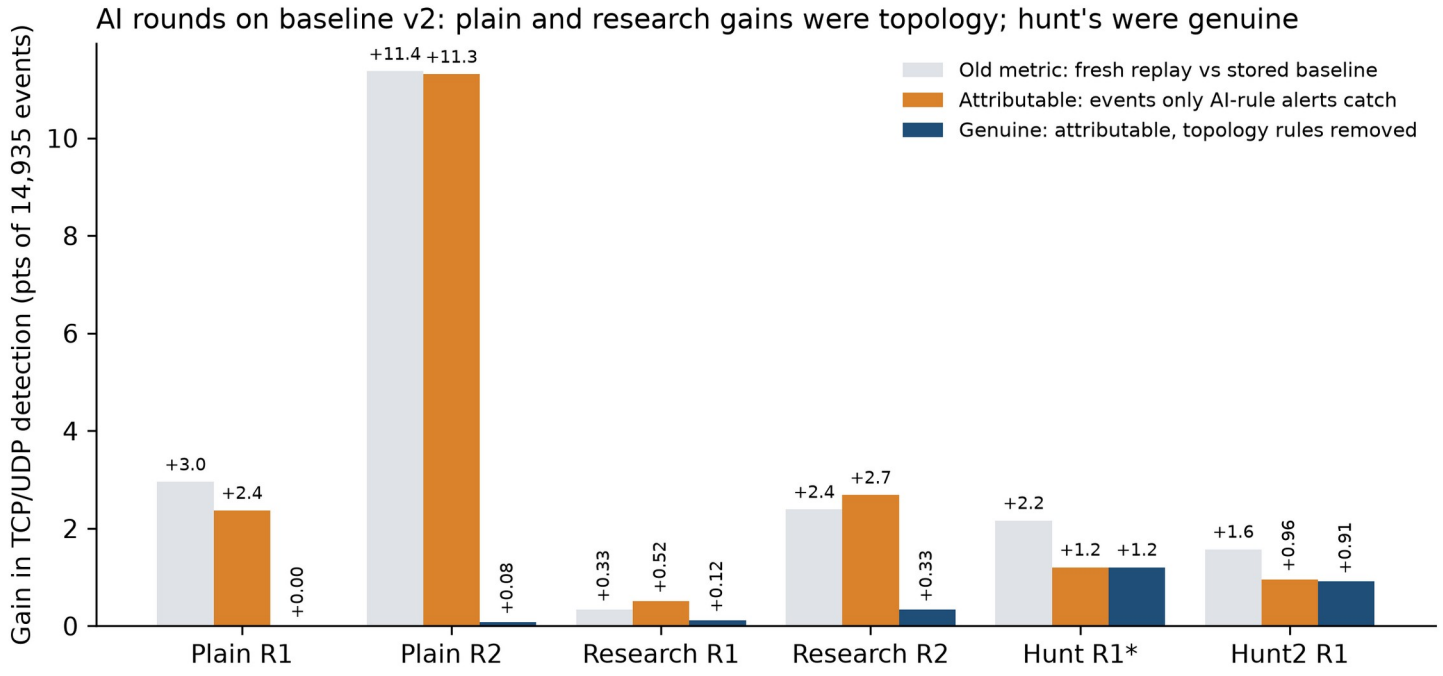
Baseline v2 detection by attack category — most attack events are still missed in every category



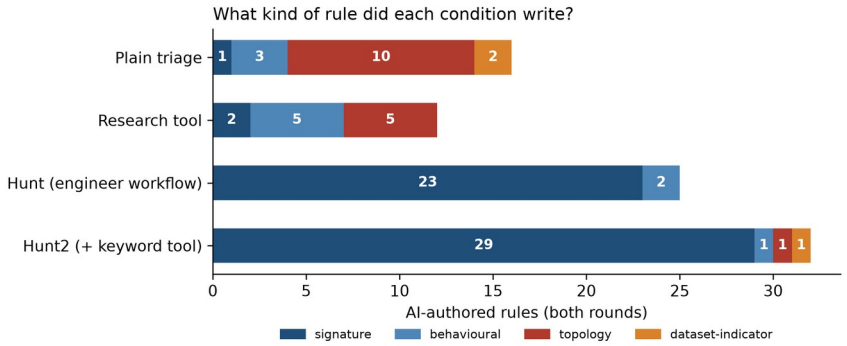

- Roughly a quarter to a third caught in the big categories: Exploits 29%, Fuzzers 32%, Reconnaissance 25%
- Weakest: Generic 7% of 2,677 events, Analysis 0%, DoS 20%
- Most missed events are one-off attempts generated with varying content — a rule written for one instance rarely matches the next

# AI rounds on the strong baseline: the topology trap

Rule-writing rounds on captures 3, 1, 10; scored on all six. Gains attributed only to events no other alert caught.




*Hunt rules written on capture 3 before the reasoning-carry fix, on captures 1 and 10 after it (Section V). Scored on all 6 dev captures.

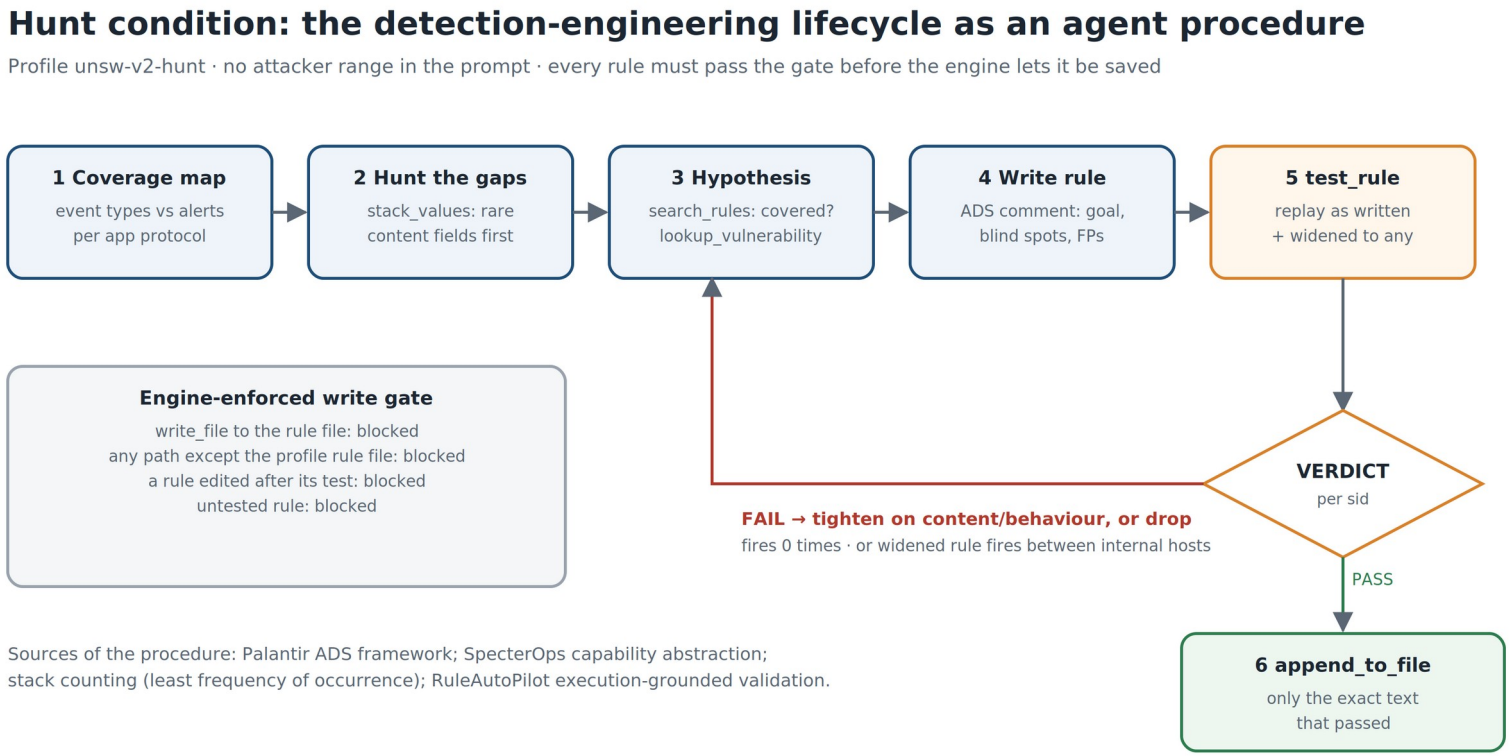



**Plain R2 looked like +11.3 pts, genuine +0.08. Research: +0.33. Hunt / hunt2: +1.21 / +0.91 — the only gains clearly above noise.**

Topology rule = ordinary activity from outside HOME_NET (SMTP MAIL FROM, POP3 login, SIP REGISTER). In UNSW every external host is an attacker, so these score perfectly here — and would alert on every customer on a real network.

# Replicating a real detection engineer

Hunt condition: the detection-engineering lifecycle, encoded as the agent's procedure




### Palantir ADS

every rule documents goal, blind spots, false positives

### SpecterOps

detect the technique, not the tool's surface strings

### Stack counting

rare values first — no ML, no training period

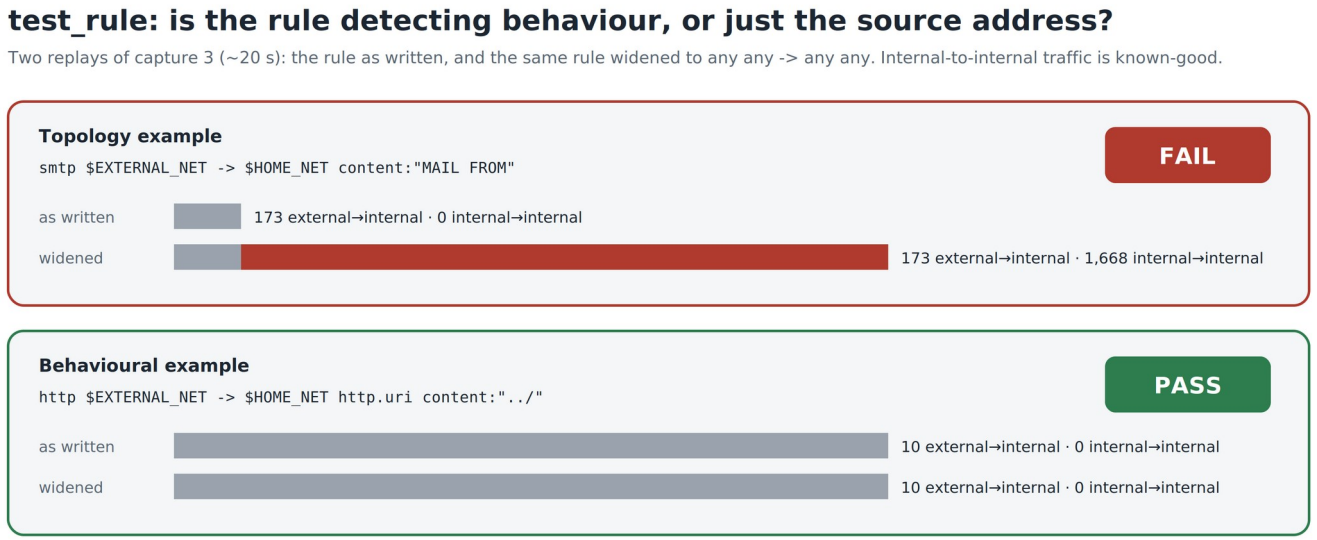
### RuleAutoPilot (2026)

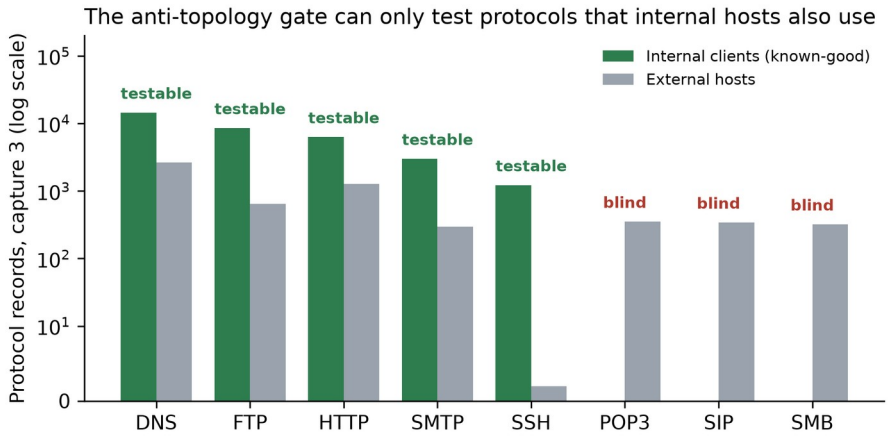
execution-grounded validation: F1 0.443 → 0.539

No attacker range in the prompt. Two new tools: stack_values (rare-first counting) and test_rule. The engine refuses to save any rule that has not passed test_rule in exactly that form.

# The anti-topology gate

Replay the rule as written, then widened to any → any. If it fires between internal hosts, it is matching normal activity.




- Known-good = internal clients → servers (testbed documentation: servers 1 and 3 generate normal traffic)
- 20 s per test on a 2 GB capture — cheap enough for the agent to iterate
- Limitation: blind on POP3, SIP and SMB — no internal host uses them in UNSW

# We found a bug by watching the agent fail

Capture 1, first attempt: 100 turns, 8.4M tokens, rules passed the gate — and nothing was saved



**Symptom**

Re-tested rules that had already passed, never saved, hit the 100-turn limit. The UI showed tool calls but no reasoning at all.



**Diagnosis**

Engine, server and browser all passed text through. The raw API did not: DeepSeek V4 Pro answered in thinking mode — reasoning in reasoning_content, content empty.



**Root cause**

Our adapter read only content. The model's reasoning was never logged and never sent back, so every turn it started again without its own plan.



**Fix**

Stream reasoning to the UI and transcript, and return it with each turn. Re-run: finished in 90 turns, 9 rules saved, 5.2M tokens.

Every earlier DeepSeek run carried the same defect whenever the model used thinking mode; most also wrote visible text, so they were affected less. Stated as a threat to validity.

# Hunt result: technique rules that transfer

25 rules from captures 3, 1, 10 — capture 3 ran before the fix, captures 1 and 10 after it

Hunt rules: alerts on each development capture (grey = the capture the rule was written on)

| | capture 1 | capture 3 | capture 4 | capture 5 | capture 10 | capture 14 |
|---|---|---|---|---|---|---|
| **written on capture 3 — before the fix** | | | | | | |
| 7800001 D-Link DIR-100 auth bypass xmlset_roodkcable | 0 | 1 | 0 | 0 | 0 | 0 |
| 7800002 HP OpenView NNM jovgraph.exe displayWidth ov | 0 | 1 | 0 | 0 | 0 | 0 |
| 7800003 HP OpenView NNM nnmRptConfig.exe format stri | 0 | 1 | 0 | 0 | 0 | 0 |
| 7800004 Cisco Secure ACS CSUserCGI.exe Logout overfl | 0 | 1 | 0 | 0 | 0 | 0 |
| 7800005 chetcpasswd.cgi access | 0 | 1 | 0 | 0 | 0 | 0 |
| 7800006 HP OpenView NNM OvWebHelp.exe Topic overflow | 0 | 1 | 0 | 0 | 0 | 0 |
| 7800007 PHP webshell creation via URL parameter | 0 | 1 | 0 | 0 | 0 | 0 |
| **written on capture 1 — after the fix** | | | | | | |
| 7800008 Unicode overlong directory traversal attempt | 1 | 0 | 1 | 0 | 0 | 0 |
| 7800009 encoded backslash traversal attempt | 1 | 0 | 0 | 1 | 0 | 1 |
| 7800010 HTTP URI format string attempt | 2 | 0 | 0 | 2 | 0 | 0 |
| 7800011 phpinfo in HTTP User-Agent | 1 | 0 | 1 | 0 | 0 | 0 |
| 7800012 XSS in HTTP User-Agent | 1 | 0 | 0 | 0 | 0 | 0 |
| 7800013 encoded XSS in HTTP URI | 5 | 7 | 9 | 6 | 5 | 1 |
| 7800014 FTP RETR path traversal attempt | 35 | 34 | 38 | 35 | 23 | 28 |
| 7800015 encoded backslash traversal attempt raw dots | 1 | 2 | 1 | 1 | 0 | 1 |
| 7800016 FTP command format string attempt | 4 | 2 | 0 | 4 | 2 | 4 |
| **written on capture 10 — after the fix** | | | | | | |
| 7800017 HP HPEinc remote file inclusion attempt | 8 | 5 | 7 | 6 | 6 | 6 |
| 7800018 Novell ZENworks UploadServlet WAR upload att | 1 | 0 | 0 | 0 | 2 | 1 |
| 7800019 HTTP path traversal attempt | 14 | 13 | 11 | 11 | 12 | 8 |
| 7800020 XSS in HTTP URI | 0 | 0 | 0 | 0 | 1 | 0 |
| 7800021 DNS BIND authors query attempt | 0 | 2 | 0 | 0 | 1 | 0 |
| 7800022 SIP INVITE with excessively long URI | 2 | 0 | 0 | 0 | 4 | 4 |
| 7800023 TLS certificate subject null byte | 1 | 0 | 0 | 0 | 1 | 1 |
| 7800024 SMTP RCPT TO format string attempt | 1 | 0 | 0 | 1 | 1 | 1 |
| 7800026 SMB NT_CREATE_ANDX fuzz filename | 36 | 0 | 12 | 36 | 12 | 36 |

**+1.21 pts**

genuine detection on the 6 dev captures (+180 events no other rule caught)

**74%**

of hunt-rule alerts match a labelled attack; 0 on normal hosts; 0 topology rules

**6 → 175**

events added by rules written before the fix (instances) vs after it (techniques)

Top contributors: SMB fuzzed filenames (behavioural, +59), HTTP /../ traversal (+37), HP HPEinc file inclusion (+31), encoded XSS (+24). The SMB rule could not be gate-tested (no internal SMB); 0 alerts on benign hold-out captures.

# Hold-out: traffic nobody had looked at

UNSW 22 Jan 2015 — rule files frozen first, then replayed and scored once




Signature and behavioural rules only; events no other alert in the same replay detected. Hunt captures 1, 10 and all hunt2 runs had the reasoning-carry fix; plain and research did not.

**+1.53 pts**

hunt rules on the hold-out (+71 of 4,634 events) — larger than on the dev set

**0**

AI-rule alerts on the two benign-only hold-out captures, in every condition

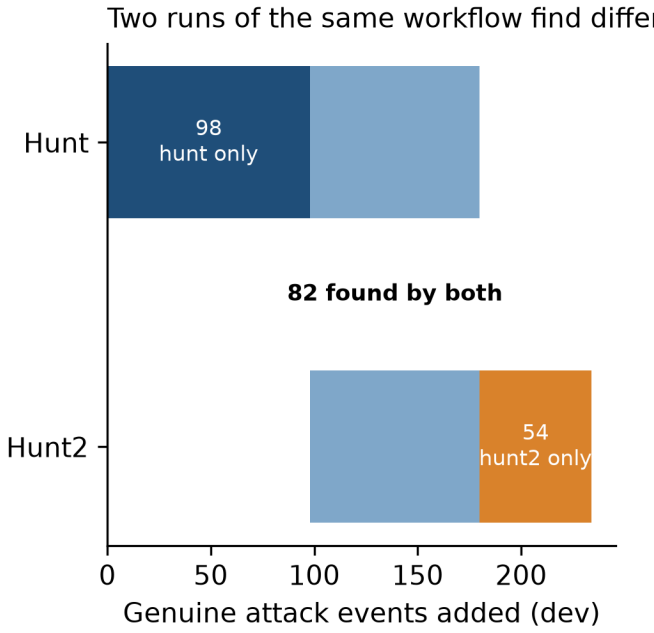
**29.5%**

detected by the frozen human baseline v2 alone (dev: 24.9%)

Hunt2: +1.06 pts. All four conditions together: about +2.2 pts. Plain and research: +0.11 and +0.28 — their large raw gains were topology rules.

# Hunt2: the keyword tool fixed errors, not detection

Same workflow plus suricata_keywords (which rule keywords this Suricata supports); all runs after the fix




**Hunt — top rules**

| | |
|---|---|
| +59 | SMB NT_CREATE_ANDX fuzz filename |
| +37 | HTTP path traversal attempt |
| +31 | HP HPEinc remote file inclusion attemp |
| +24 | encoded XSS in HTTP URI |

**Hunt2 — top rules**

| | |
|---|---|
| +36 | WebDAV PROPFIND for Windows DLL |
| +29 | HPE plugin remote file inclusion attem |
| +23 | HTTP URI directory traversal |
| +12 | Encoded obfuscated XSS with JavaScript |

## 0

rules that failed to load (hunt capture 10: 2); proper buffers instead of raw-byte regexes

## +0.91 / +0.92 / +1.06

genuine pts: dev / dev 4-5-14 / hold-out (hunt: +1.21 / +1.38 / +1.53)

## 82 shared

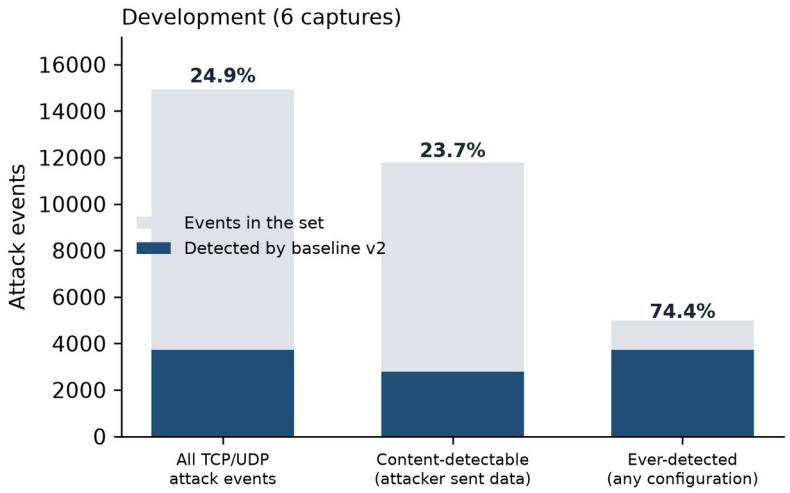
of 180 (hunt) and 136 (hunt2) events: runs of the same workflow find different attacks

Expected beforehand: +1.2 to +2.0 pts. Got less. What a run detects depends on which attack families it chooses to hunt — run-to-run variation is larger than the tool's effect.

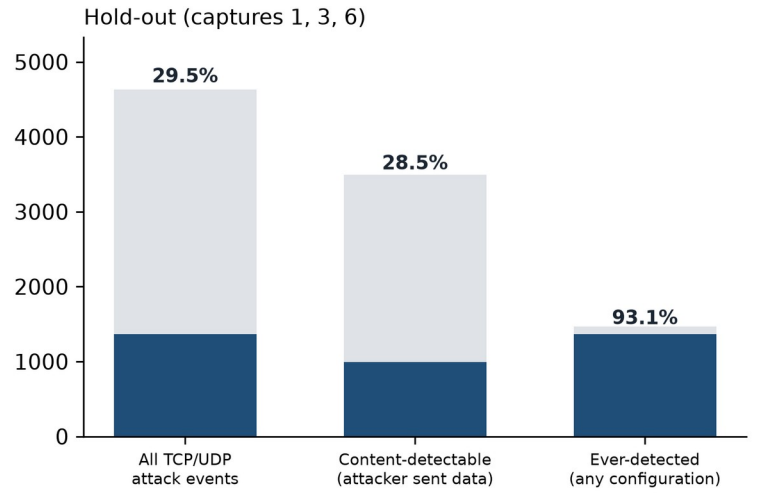
# Is 25% unfair to a signature IDS?

Measured against attacks a rule could plausibly see — not every labelled event

Most missed attacks carried data: the gap is missing signatures, not undetectable traffic




Genuine AI gain, pts of ever-detected:
plain +0.24 research +1.00 hunt +3.60 hunt2 +2.72




Genuine AI gain, pts of ever-detected:
plain +0.34 research +0.88 hunt +4.83 hunt2 +3.34

**79%**

of attack events carried data a content rule could see

**23.7%**

of those the tuned baseline detects — the gap is unwritten signatures

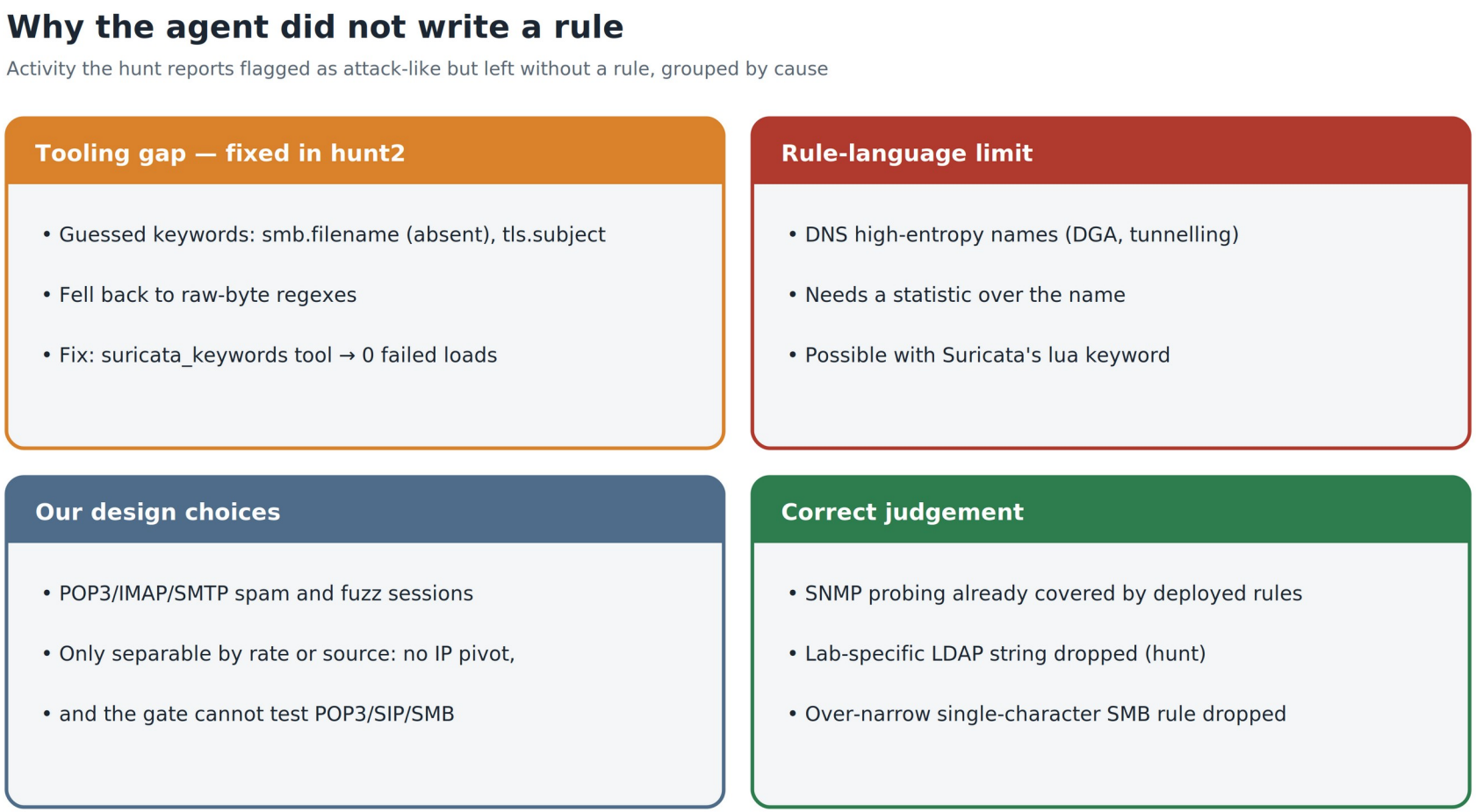
**+4.8 pts**

hunt gain measured against everything any configuration caught (hold-out)

Content-detectable: attacker sent more bytes than headers account for (PSH flag unusable: the generator omits it). Ever-detected: caught by any configuration we ran — a circular ceiling.

# Why the agent did not write a rule

Attack-like activity the hunt reports left without a rule, grouped by cause



# Conclusions

**Answer — a split claim:** yes for triage: with sensor context the model investigates end to end (13/14 CIC, 8/9 UNSW categories) and establishes outcomes. Partly for detection: against a strong human-tuned baseline, triage-driven rules added a fraction of a point; the detection-engineering workflow, with the model's reasoning carried across turns, added +1.2–1.5 pts (hunt2: ~+1.0) that held on unseen traffic with no false alarms — real, but modest against the ~70% still missed.

### Tools and procedure shape the agent

count_by fixed a failing model; the hunt procedure removed topology rules

### Carry the model's reasoning

Dropping it made the agent loop; returning it fixed the hunt

### Measure what the AI alone caught

Replay variance and topology rules both inflated our early numbers

### Techniques transfer, instances don't

Per-exploit rules fired once; traversal, XSS and fuzzing rules held on the hold-out

# Next steps

### Separate workflow from fix

Re-run plain and research with reasoning carried, to isolate what the hunt workflow adds

### Technique-level hunting

Require technique rules and test each on a second capture before saving

### Several runs per capture

Runs find different attacks: keep the union, measure the spread

### Live deployment

Time-windowed alert batches and live rule reload via the Suricata socket



# Questions?

Code, rules, transcripts and evaluation scripts:
github.com/bigyan-siwakoti/Intrusion_Detection_System